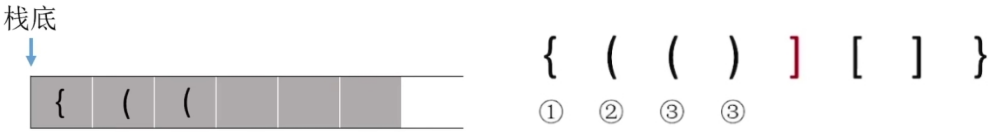
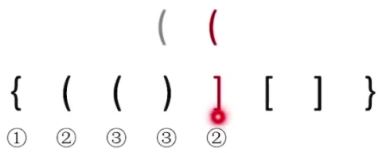
§ 3.3 栈的应用

一、栈在括号匹配中的应用

原则：遇到一个右括号，就消耗一个左括号，不匹配则失败



可能遇到的问题：



不匹配

单身右括号

单身左括号，剩在栈中

算法实现（利用 LIFO 特性）

```
bool bracketCheck(char str[], int length) {
    SqStack S;
    InitStack(S);
    for (int i = 0; i < length; i++) {
//开始扫描，结果一定是得到左/右括号
        if (str[i] == '(' || str[i] == '[' || str[i] == '{') {
            Push(S, str[i]);
//扫描到左括号，压入栈中
        } else {
            if (StackEmpty(S)) return false;
            char topElem;
            Pop(S, topElem);
//扫描到右括号，栈顶元素出栈
            if (!match(topElem, str[i])) return false;
//不匹配则停止扫描
        }
    }
    return StackEmpty(S);
}
```

二、栈在表达式求值中的应用

1. 表达式的三种形式

类型	示例		
中缀表达式	a + b * c	a+b-c*d	a+b-c
后缀表达式（逆波兰）	a b c * +	ab+ cd* -	后缀：a bc- +或者 ab+ c-
前缀表达式（波兰）	+ a * b c	- +ab *cd	

2. 中缀转后缀（手算）

((15 ÷ (7 − (1 + 1))) × 3) − (2 + (1 + 1))

③ ② ① ④ ⑦ ⑥ ⑤

11+ 11+

7 11+ - 2 11+ +

15 7 11+ - ÷

15 7 11+ - ÷ 3*

后缀表达式的符号顺序是加减乘除，和中缀表达式的运算顺序相同

运算符必须写在操作数之后，所以 3*不能写在 15 前面

算法一定具有确定性，为了保证手算结果与机算相同，引入“左优先”规则

A + B * (C - D) – E / F　　A + B * (C - D) – E / F

③ ② ① ⑤ ④　　⑤ ③ ② ④ ①

✓ ① ② ③ ④ ⑤
A B C D - * + E F / -

✗ ② ③ ① ④ ⑤
A B C D - * E F / - +

私房菜：“左优先”原则，不要FreeStyle，保证手算和机算结果相同

“左优先”原则：只要左边的运算符能先计算，就优先算左边的

客观来看两种都正确，是“机算”结果是前者

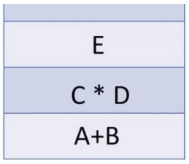
eg.E/F 左边的 C-D 可以先生效，就先算减法

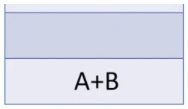
4. 后缀机算

- 初始化一个**操作数栈**
- 从左到右扫描：
    - 操作数 → 入栈
    - 运算符 → 弹出两个操作数，结果压回

注意：<u>先弹出</u>的是<u>右操作数</u>

A + B - C * D / E + F
① ④ ② ③ ⑤





(C*D) / E

5. 中缀转前缀（手算）

和中缀转后缀类似，遵循“右优先”原则

6. 前缀机算

- 初始化**操作数栈**和**运算符栈**
- 从右到左扫描：
    - 操作数 → 入操作数栈
    - 运算符 → 弹出两个操作数，结果压回

注意：<u>先弹出</u>的是<u>左操作数</u>

- × ÷ 15 - 7 + 1 1 3 + 2 + 1 1
⑦ ⑥ ⑤ ④ ③ ② ①

7. 中缀转后缀（计算）

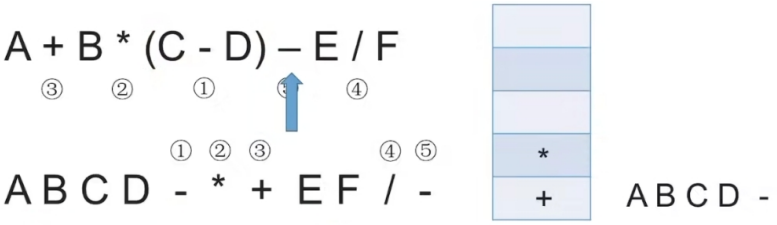
从左到右处理各个元素，直到末尾。可能遇到三种情况：

① 遇到操作数。直接加入后缀表达式。

② 遇到界限符。遇到“(”直接入栈；遇到“)”则依次弹出栈内运算符并加入后缀表达式，直到弹出“(”为止。注意：“(”不加入后缀表达式。

③ 遇到运算符。依次弹出栈中优先级高于或等于当前运算符的所有运算符，并加入后缀表达式，若碰到“(”或栈空则停止。之后再把当前运算符入栈。

按上述方法处理完所有字符后，将栈中剩余运算符依次弹出，并加入后缀表达式。

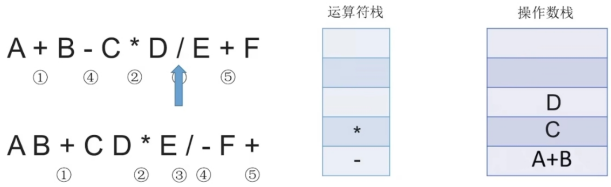


8.用栈实现中缀表达式求值（中缀转后缀+后缀机算）

初始化两个栈；

从左往右，操作数压入操作数栈；

界限符或运算符，按照中缀转后缀的原则，判断是否要压入/弹出；



三、栈在递归中的应用

1. 函数调用过程

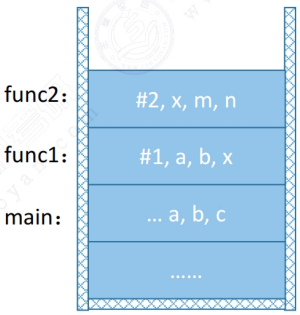
```
void main() {
  int a, b, c;
  ...
  func1 (a, b);
#1 c=a+b;
  ...
}
```

```
void func1 (int a, int b) {
  int x;
  ...
  func2 (x);
#2 x=x+10086;
  ...
}
```

```
void func2 (int x) {
  int m, n;
  ...
}
```

函数调用的特点：最后被调用的函数最先执行结束（LIFO）

函数调用时，需要用一个栈存储：
① 调用返回地址
② 实参
③ 局部变量



函数调用栈

最后被调用的函数最先返回（LIFO）

2. 递归调用的栈

适合用“递归”算法解决：可以把原始问题转换为属性相同，但规模较小的问题

Eg 1：计算正整数的阶乘 n!

$$\text{factorial}(n) = \begin{cases} n * \text{factorial}(n-1), & n>1 \\ 1, & n=1 \\ 1, & n=0 \end{cases}$$

递归表达式（递归体）

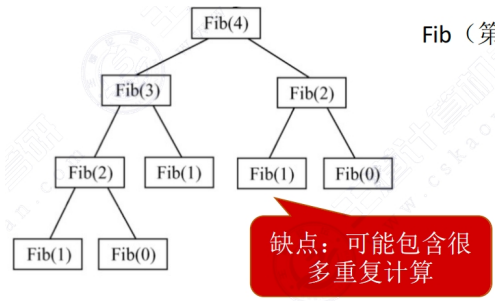
边界条件（递归出口）

- 每进入一层递归 → 压栈
- 每退出一次递归 → 出栈
- 递归深度过大 → **栈溢出**

3. 递归示例：斐波那契数列

```
int Fib(int n){
    if(n==0)
        return 0;
    else if(n==1)
        return 1;
    else
        return Fib(n-1)+Fib(n-2);
```

在下一个循环中，n=1 和 n=0 会导致 return1 和 return0，并把 1，0 返回给上一层，上一层再向上返回



Fib（第

四、队列的应用

此部分内容在图、树章节详细学习，在此不整理内容